SIMATS ENGINEERING
ASSESSMENT TOOL 2: Reverse Engineering

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: CSA17

Assessment Tool Weightage: 30% **N.sri vara deepak**

Code of Conduct: **192572075**

I **N.sri vara Deepak(192572075)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: **Deepak.N**

1. System Decomposition & Analysis

Analyzes an existing AI-based system and **reverse engineer its architecture**, identifying key components, data flow, and underlying algorithms.

2. Architecture Reconstruction

Given a black-box software system, **reconstruct its system architecture** and explain the interaction between modules using appropriate diagrams.

3. Algorithm Identification

Reverse engineer a machine learning application to **identify the algorithms, data processing steps, and decision-making logic** used in the system.

4. Data Flow & Process Mapping

Examine an existing distributed AI application and **map its data flow, processing pipeline, and communication between components**.

5. Improvement & Redesign

Perform reverse engineering on an intelligent system and **propose enhancements or redesign strategies** to improve scalability, efficiency, and performance.

Reverse Engineering of AI Systems

1. System Decomposition & Analysis

System: AI-Based Healthcare Diagnosis System

Objective:
To analyze patient information and predict possible diseases using machine learning.

Main Components:

1. **User Interface**
    - Doctors or patients enter symptoms and medical information.
    - Displays prediction and recommendations.
2. **Data Collection Module**
    - Collects age, symptoms, blood pressure, glucose level, etc.
    - Receives data from forms or medical devices.
3. **Data Preprocessing**
    - Removes missing or incorrect values.
    - Converts categorical data into numerical form.
    - Normalizes numerical features.
4. **Machine Learning Model**
    - Uses algorithms such as Decision Tree, Random Forest, or Neural Network.
    - Learns patterns from historical patient data.
5. **Prediction Module**

- Receives processed patient data.
- Predicts the possible disease.

6. **Database**
   - Stores patient records, previous diagnoses, and prediction results.
7. **Result Module**
   - Displays the predicted disease and confidence score.

Data Flow:

Patient Data

↓

User Interface

↓

Data Collection

↓

Preprocessing

↓

ML Model

↓

Prediction

↓

Result Display

↓

Database

Underlying Algorithms:

- Decision Tree
- Random Forest
- Logistic Regression
- Neural Network

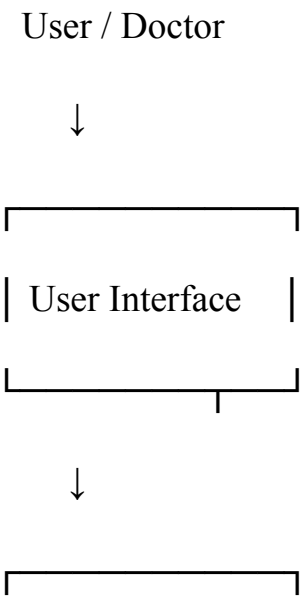
Conclusion:
Reverse engineering divides the healthcare AI system into smaller modules and helps understand how data moves from input to prediction.

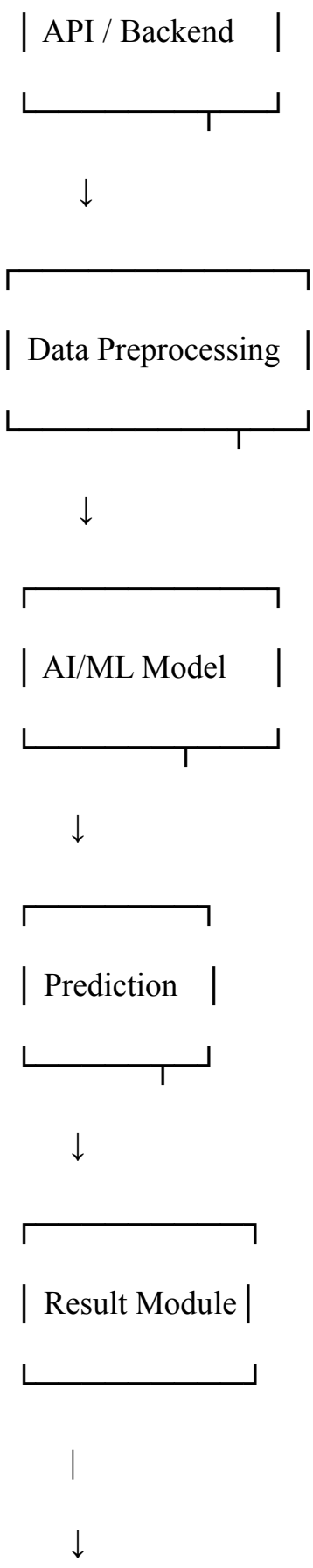
2. Architecture Reconstruction

System: Black-Box AI Healthcare Application

When the internal implementation is unknown, the system can be reconstructed by observing its inputs, outputs, modules, and interactions.

Reconstructed Architecture




API / Backend
Data Preprocessing
AI/ML Model
Prediction
Result Module

Database

Module Interaction

1. **User Interface → Backend**
    - Sends patient information.
2. **Backend → Preprocessing**
    - Validates and prepares the input data.
3. **Preprocessing → ML Model**
    - Sends cleaned numerical features.
4. **ML Model → Prediction**
    - Generates disease prediction.
5. **Prediction → Result Module**
    - Converts model output into understandable information.
6. **Database**
    - Stores patient information and prediction history.

Advantages of Architecture Reconstruction

- Helps understand an unknown system.
- Identifies dependencies between modules.
- Helps locate performance bottlenecks.
- Makes maintenance easier.
- Supports future redesign.

Conclusion:
Architecture reconstruction provides a logical representation of a black-box system even when its original source code is unavailable.

3. Algorithm Identification

System: Machine Learning-Based Disease Prediction

Reverse engineering can be used to identify the algorithms and processing steps used by an AI application.

Step 1: Data Input

Patient data may contain:

- Age
- Blood pressure
- Glucose level
- BMI
- Cholesterol
- Family history
- Symptoms

Step 2: Data Preprocessing

The system performs:

Raw Data

↓

Missing Value Handling

↓

Data Cleaning

↓

Feature Encoding

↓

Feature Scaling

↓

Training Data

Step 3: Feature Selection

Important features are selected based on their contribution to prediction.

For example:

Glucose

Blood Pressure

BMI

Age

↓

Feature Selection

↓

Important Features

Step 4: Algorithm Identification

Possible algorithms can be identified by analyzing system behavior.

Decision Tree:

- Uses conditions such as:
    - Glucose > threshold?
    - BMI > threshold?
    - Age > threshold?
- Produces a tree-like decision structure.

Random Forest:

- Uses multiple decision trees.
- Combines their predictions.
- Generally provides more robust predictions than a single tree.

Neural Network:

- Uses interconnected layers.
- Learns complex relationships between features.

Step 5: Decision-Making Logic

Example:

Glucose Level

↓

Is glucose high?

/ \

Yes No

↓ ↓

Check BMI Low Risk

↓

Prediction

Reverse Engineering Table

Component	Identified Operation
Input	Patient information
Preprocessing	Cleaning and normalization
Feature Selection	Select important features
Learning	ML algorithm
Prediction	Disease classification
Output	Disease/risk category

Conclusion:
Algorithm identification helps determine how an AI application transforms raw data into a final decision.

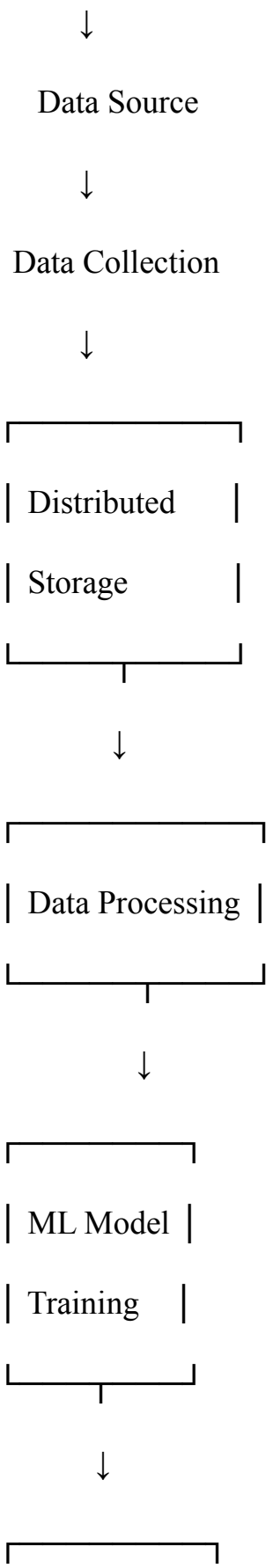
4. Data Flow & Process Mapping

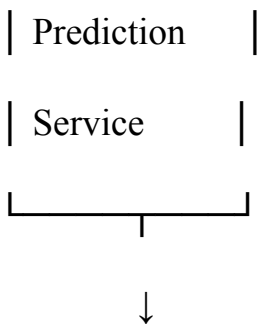
System: Distributed AI Healthcare Application

A distributed AI system processes data across multiple components instead of using a single computer.

Data Flow

Patient / Hospital


Data Source
Data Collection
Distributed
Storage
Data Processing
ML Model
Training



Doctor / Patient

Processing Pipeline

1. Data Generation

- Patient records are generated by hospitals, sensors, or applications.

2. Data Collection

- Data is transferred to the distributed system.

3. Data Storage

- Large datasets are stored in distributed storage.

4. Data Processing

- Data is cleaned and transformed using distributed processing.

5. Model Training

- Machine learning algorithms process large datasets.

6. Prediction

- The trained model receives new patient data.

7. Result Delivery

- Prediction is sent to the doctor or user.

Communication Between Components

Component	Communication
User → Server	API/HTTP
Server → Database	Database connection
Storage → Processing	Distributed data transfer
Processing → ML Model	Data pipeline
ML Model → User	API response

Benefits

- Handles large datasets.
- Supports parallel processing.
- Improves scalability.
- Reduces processing time.
- Provides fault tolerance.

Conclusion:
Data-flow mapping clearly shows how information travels through a distributed AI system from data generation to final prediction.

5. Improvement & Redesign

System: Existing AI Healthcare Diagnosis System

After reverse engineering the system, several improvements can be proposed.

Existing System

User

↓

Server

↓

Database

↓

ML Model

↓

Prediction

Problems Identified

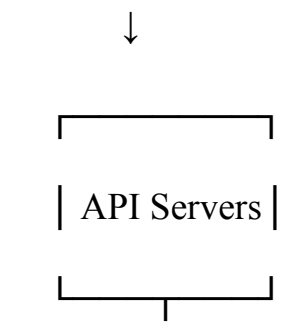
1. Large datasets may slow down processing.
2. Single server can become a bottleneck.
3. Model prediction may take longer with increasing users.
4. Centralized storage can create scalability problems.
5. Model performance may decrease when new data becomes available.

Proposed Redesign

Users

↓

Load Balancer



↓

Data Processing

↓

Distributed Storage

↓

ML Model Service

↓

Prediction

↓

Result / Dashboard

Improvements

1. Distributed Processing

- Divide large datasets into smaller parts.
- Process them simultaneously.

2. Cloud-Based Storage

- Store large amounts of healthcare data efficiently.

- Increase storage when required.

3. Model Optimization

- Remove unnecessary features.
- Use efficient ML algorithms.
- Reduce prediction time.

4. Load Balancing

- Distribute user requests across multiple servers.
- Prevent one server from becoming overloaded.

5. Model Updating

- Retrain the model periodically using new data.
- Improves prediction performance.

6. Caching

- Store frequently requested results.
- Reduces repeated computation.

7. Monitoring

- Monitor CPU, memory, response time, and model accuracy.
- Detect failures quickly.

Comparison

Feature	Existing System	Redesigned System
Processing	Centralized	Distributed
Storage	Limited	Scalable
Performance	Moderate	Improved
Scalability	Low	High
Fault Tolerance	Low	High
Model Updates	Manual	Periodic/automated
User Load	Limited	High

Conclusion

Reverse engineering helps identify weaknesses in an existing AI system. The redesigned architecture uses **distributed processing, scalable storage, load balancing, caching, and model optimization** to improve **performance, scalability, reliability, and efficiency**.

RUBRICS FOR CALCULATION:

Evaluation Criteria	System Understanding	Decomposition & Analysis	Reconstruction of Architecture	Identification of Algorithms/Logic	Use of Tools & Techniques	Data Flow & Process Mapping	Problem-Solving & Interpretation	Innovation & Improvement Suggestions	Documentation & Presentation
Weightage	15%	15%	15%	10%	10%	10%	10%	5%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
System Understanding	Clearly identifies system purpose, components, and functionality	Mostly clear understanding	Basic understanding	Poor or incorrect
Decomposition & Analysis	Accurately breaks system into modules/components	Good decomposition	Partial breakdown	Incomplete analysis
Reconstruction of Architecture	Recreates system architecture with clear diagrams	Good reconstruction	Basic structure	Poor/incomplete
Identification of Algorithms/Logic	Clearly identifies underlying algorithms and logic	Mostly correct	Limited identification	Incorrect/missing
Use of Tools & Techniques	Effectively uses reverse engineering tools/techniques	Good usage	Limited use	No proper use
Data Flow & Process Mapping	Clearly explains data flow and interactions	Mostly clear	Some gaps	Poorly defined
Problem-Solving	Provides deep	Good	Basic insights	Weak analysis

& Interpretation	insights and correct interpretations	interpretation		
Innovation & Improvement Suggestions	Suggests meaningful improvements/redesign	Some suggestions	Basic ideas	No suggestions
Documentation & Presentation	Well-structured, clear diagrams and explanation	Mostly clear	Somewhat organized	Poor presentation